

附录核心源代码

正文后附：B* 树 / Skyline 解码 / 扰动 / 快速模拟退火 / 二分与双向确认 / 超块穷举

2026 年第七届”华数杯”大学生数学建模竞赛 B 题

2026/08/10

本附录列出与正文模型、算法对应的**核心函数源代码**（与提交源程序逐行一致），供代码复核与结果复现时对照。

1 节点数据结构（10-bit 位压缩）

B* 树节点将父指针、左右子指针与旋转位压缩进一个 32 位整数，单节点仅 4 字节，支持至 1023 节点。

```
1
2 template<typename ID, typename LEN> struct FLOOR_PLAN<ID, LEN>::NODE {
3     NODE() : x(0) {};
4     ID _l() const { return (x >> 01) & ((1 << 10) - 1); }
5     ID _r() const { return (x >> 11) & ((1 << 10) - 1); }
6     ID _p() const { return (x >> 21) & ((1 << 10) - 1); }
7     bool _rot() const { return x & 1; }
8     void s_l(const uint& i) { x = ((x & rmask_l) | ((i << 01) & mask_l)); }
9     void s_r(const uint& i) { x = ((x & rmask_r) | ((i << 11) & mask_r)); }
10    void s_p(const uint& i) { x = ((x & rmask_p) | ((i << 21) & mask_p)); }
11    void s_rot() { x ^= 1; }
12    unsigned int x;
13 };
```

2 Skyline 轮廓压缩解码

按先序遍历放置：左子紧贴父右、右子与父左对齐， y 取轮廓区间最大顶高；任意树解码结果天然无重叠，摊还复杂度 $O(n)$ 。

1

```

2 void dfs(ID id, list<ID>& cy, typename list<ID>::iterator& cur,
3         vector<BLOCK>& blcks) {
4     if (_nodes[id]._l()) {
5         const ID& lc = _nodes[id]._l();
6         blcks[lc]._x = blcks[id]._x + blcks[id]._w;
7         blcks[lc]._y = find_max_y(cy, ++cur, blcks, blcks[lc]);
8         dfs(_nodes[id]._l(), cy, cur, blcks);
9         --cur;
10    }
11    if (_nodes[id]._r()) {
12        const ID& rc = _nodes[id]._r();
13        blcks[rc]._x = blcks[id]._x;
14        blcks[rc]._y = find_max_y(cy, cur, blcks, blcks[rc]);
15        dfs(_nodes[id]._r(), cy, cur, blcks);
16    }
17 }
18 LEN find_max_y(list<ID>& l, typename list<ID>::iterator& cur,
19               vector<BLOCK>& blcks, BLOCK& blk) {
20     LEN y = 0;
21     auto it = cur, rit = cur;
22     while (it != l.end() && blcks[*it]._x < blk._x + blk._w) {
23         if (blcks[*it]._y + blcks[*it]._h > y) y = blcks[*it]._y + blcks[*
24             it]._h;
25         if (blcks[*rit]._x + blcks[*rit]._w <= blk._x + blk._w) ++rit;
26         ++it;
27     }
28     cur = l.erase(cur, rit);
29     cur = l.insert(cur, blk._id);
30     return y;
31 }

```

3 三种扰动算子

每步以 0.30 / 0.35 / 0.35 的概率执行旋转、删除-插入或节点交换，三者均保持树拓扑合法（节点集合守恒、无环）。

```

1
2 void perturb() {
3     float p1 = randf(), p2 = randf();
4     if (p1 < _rot_prob) rotate();
5     else if (p2 < _del_and_ins_prob) del_and_ins();
6     else swap_two_nodes();
7     _has_init = false;
8

```

```

9
10 void rotate() {
11     if (_rotable.empty()) return;
12     ID id = _rotable[(rand() % _rotable.size())];
13     _blcks[id]._rot = !_blcks[id]._rot;
14     swap(_blcks[id]._w, _blcks[id]._h);
15     _tree.rotate(id);
16 }
17 void del_and_ins() {
18     if (_Nblcks < 2) return;
19     ID id = (rand() % _Nblcks) + 1;
20     _tree.del_from_tree(id);
21     ID p = (rand() % _Nblcks) + 1;
22     bool left = randb();
23     while (id == p) p = (rand() % _Nblcks) + 1;
24     _tree.ins_to_tree(p, id, left);
25 }
26 void swap_two_nodes() {
27     if (_Nblcks < 2) return;
28     ID id1 = rand() % _Nblcks;
29     ID id2 = (id1 + ((rand() % (_Nblcks - 1)) + 1)) % _Nblcks;
30     _tree.swap_two_nodes(id1 + 1, id2 + 1);
31 }

```

4 代价函数与可行性判定

问题一以 λ 加权面积与长宽比对称惩罚；问题二 run 阶段用可行搜索代价（面积 + 轮廓压力），run2 阶段用精修代价；可行性判定为外接矩形不越出固定轮廓。

```

1
2 float norm_cost(const int3& cost) const {
3     const float r = float(get<2>(cost)) / get<1>(cost);
4     const float area_term = _alpha * get<1>(cost) * get<2>(cost) /
5         _avg_area;
6     const float hpwl_term = (_avg_hpwl > 0) ? _beta * get<0>(cost) /
7         _avg_hpwl / 2. : 0.f;
8     const float r_term = (_avg_r > 0)
9         ? (1 - _alpha - _beta) * (r - _R) * (r - _R) / _avg_r : 0.f;
10    return area_term + hpwl_term + r_term;
11 }
12 float true_cost(const int3& cost, const float den = 1) const {
13     return (_true_alpha * get<1>(cost) * get<2>(cost)
14         + (1 - _true_alpha) * get<0>(cost) / 2.) / den;
15 }

```

```

14 bool feas(const int3& cost) {
15     if (_mode == Mode::Q1) return true;
16     return (get<1>(cost) <= _W && get<2>(cost) <= _H);
17 }

```

5 快速模拟退火（精修阶段 run2 主循环）

低温精修主循环：扰动、解码、Metropolis 接受判断、自适应降温与停滞重置（re-heating）；best 仅更新于可行且更优的解。

```

1  run2(const int k, int rnd, const float c) {
2      float _init_T2 = _init_T / ((_mode == Mode::Q1) ? 20.f : 50.f);
3      int reset_th = 2 * _Nblcks, stop_th = 9 * _Nblcks, reset_cnt = 0;
4      int iter = 1, tot_feas = 0, rej_num = 0, cnt = 1;
5      _fp.init();
6      tot_feas = feas(_fp.cost()) ? 1 : 0;
7      float _T = _init_T2;
8      float prv_cost = (_mode == Mode::Q1) ? q1_cost(_fp.cost()) :
9          true_cost(_fp.cost(), _avg_true);
10     _best_cost = prv_cost;
11 #ifdef TREE_DEBUG
12     _dbg_hist.push_back(-1.0f);
13 #endif
14     dbg_push(_best_cost);
15     typename FLOOR_PLAN<ID, LEN>::TREE last_sol = _best_sol;
16     while (_T > temp_th || float(rej_num) <= rej_ratio * cnt || !tot_feas
17         ) {
18         float avg_delta_cost = 0;
19         rej_num = 0, cnt = 1;
20         for (; cnt <= rnd; ++cnt) {
21             _fp.perturb();
22             _fp.init();
23             int3 costs = _fp.cost();
24             float cost = (_mode == Mode::Q1) ? q1_cost(costs) : true_cost(
25                 costs, _avg_true);
26             float delta_cost = (cost - prv_cost);
27             avg_delta_cost += abs(delta_cost);
28
29             if (delta_cost <= 0 || randf() < expf(-delta_cost / _T)) {
30                 prv_cost = cost;
31                 last_sol = _fp.get_tree();
32                 if (feas(costs)) {
33                     ++tot_feas;

```

```

32         if (cost < _best_cost) {
33             _best_sol = _fp.get_tree();
34             _best_cost = cost;
35             dbg_push(_best_cost);
36         }
37     }
38 } else {
39     _fp.restore(last_sol);
40     ++rej_num;
41 }
42 }
43 ++iter;
44 if (iter <= k) _T = _init_T2 * avg_delta_cost / cnt / iter / c;
45 else _T = _init_T2 * avg_delta_cost / cnt / iter;
46 _fp.init();
47 dbg_log("run2", iter, _T, _alpha, tot_feas ? 1 : 0);
48 if (_log) {
49     _snap_iters.push_back(iter);
50     _snap_trees.push_back(_best_sol);
51 }
52 if (reset_cnt > _Nblcks / 7 + 1) break;
53 if (!tot_feas) {
54     if (iter > reset_th) {
55         _T = _init_T2;
56         iter = 1;
57         ++reset_th, ++stop_th, ++rnd, ++reset_cnt;
58     }
59     } else if (iter > stop_th) break;
60 }
61 _fp.restore(_best_sol);
62 _fp.init();
63 dbg_snapshots();
64 int3 costs = _fp.cost();
65 _best_cost = (_mode == Mode::Q1) ? q1_cost(costs) : true_cost(costs);
66 if (_mode == Mode::Q1) dbg_push(q1_cost(costs));
67 else dbg_push(true_cost(costs, _avg_true));
68 return {_best_cost, _best_sol};
69 }

```

6 求解入口

两轮独立精修退火各返回最优解，取代价更小者输出。

```

2         ostream* log = nullptr, bool feas_only = false) {
3     FLOOR_PLAN<ID, LEN> fp(fnets, fblcks, fpl, rpt, Nnets, Nblcks, Ntrmns,
4         alpha, dead_ratio, mode == Mode::Q1);
5     float P = 0.9f, alpha_base = 0.5f, beta = 0.1f;
6     const float R = fp.R();
7     int k = max(2, Nblcks / 11), rnd = 2 * Nblcks + 20;
8     float c = max(100 - int(Nblcks), 10);
9     SA<ID, LEN> sa(fp, Nblcks, fp.W(), fp.H(), R, P, alpha_base, beta,
10         alpha,
11         mode, log);
12     if (feas_only) {
13         sa.run(k, rnd, c, true);
14         fp.init();
15         int3 costs = fp.cost();
16         ofstream outs(rpt);
17         outs << (sa.last_feasible() ? 1 : 0) << '\n';
18         outs << get<1>(costs) << " " << get<2>(costs) << '\n';
19         outs << get<0>(costs) / 2. << '\n';
20         return;
21     }
22     typename FLOOR_PLAN<ID, LEN>::TREE trees[2];
23     float costs[2];
24     if (mode == Mode::Q1) {
25         for (int i = 0; i < 2; ++i) {
26             tie(costs[i], trees[i]) = sa.run2(k, rnd, c);
27         }
28     } else {
29         for (int i = 0; i < 2; ++i) {
30             sa.run(k, rnd, c);
31             tie(costs[i], trees[i]) = sa.run2(k, rnd, c);
32         }
33     }
34     fp.restore(costs[0] < costs[1] ? trees[0] : trees[1]);
35     fp.init();
36     ofstream outs(rpt);
37     fp.output(outs);

```

7 问题三：二分搜索与双向确认

并行多种子判定（任一可行即可行）抑制假阴性；二分收敛后做双向确认： d^* 处可行且 $d^* - \epsilon$ 处全部种子不可行。

```

1
2 def bisect(chip, total, work_dir, trace_path, eps=EPS,

```

```

3         coarse_seeds=COARSE_SEEDS, precise_seeds=PRECISE_SEEDS):
4     """二分 [D_LO, D_HI], <eps 早停。返回 (d_star, 迭代步数)。"""
5     lo, hi = D_LO, D_HI
6     rows = []
7     it = 0
8     while hi - lo >= eps:
9         it += 1
10        mid = (lo + hi) / 2.0
11        n_seeds = coarse_seeds if (hi - lo) >= PRECISE_THRESHOLD else
            precise_seeds
12        t0 = time.time()
13        feasible, results = feas_check_parallel(chip, mid, total,
            work_dir,
14                                                    n_seeds, it)
15        dt = time.time() - t0
16        bb = results[0][1]
17        if feasible:
18            hi = mid
19        else:
20            lo = mid
21        rows.append([it, lo, hi, mid, side_for(mid, total),
22                    f"{bb[0]}x{bb[1]}", int(feasible), n_seeds, round(dt
23                        , 1)])
24    with open(trace_path, "w", newline="") as f:
25        w = csv.writer(f)
26        w.writerow(["iter", "lo", "hi", "mid", "side", "bbox", "feasible"
27            ,
28            "attempts", "time_s"])
29        w.writerows(rows)
30    return hi, it
31
32def confirm_minimum(chip, d_star, total, work_dir, verify_seeds=
33    VERIFY_SEEDS):
34    """双向确认: d* 本身必须可行 (不可行则上移 +EPS 重验);
35    d*-EPS 处多种子必须全不可行 (有可行则下移重验)。
36    返回 (final_d, 确认记录)。"""
37    steps = []
38    for k in range(12):
39        feas_here, r_here = verify_at(chip, d_star, total, work_dir, 900
40            + k,
41                                                    n_seeds=verify_seeds)
42        if not feas_here:
43            d_star = min(D_HI, d_star + EPS)
44            steps.append({"d": round(d_star, 6), "d_feas": False,

```

```

42         "note": "up-shift"})
43     continue
44     below = max(0.0, d_star - EPS)
45     feas_below, r_below = verify_at(chip, below, total, work_dir, 950
        + k,
46                                     n_seeds=verify_seeds)
47     steps.append({"d": round(d_star, 6), "below": round(below, 6),
48                 "d_feas": True, "below_infeas": not feas_below,
49                 "below_results": [r[0] for r in r_below]})
50     if not feas_below:
51         return d_star, steps
52     d_star = below
53     return d_star, steps

```

8 问题四：超块穷举（矩形分解与凹角支撑解码）

L/T 模块经矩形分解为超块，放置时以内部矩形分别对轮廓求支撑（凹角精确支撑），再做矩形级重叠检查；穷举 86 016 组合。

```

1
2 def rotate_90(rects):
3     """模块整体逆时针旋转 90 度（角点变换 + 归一化平移）。"""
4     new = []
5     for (x, y, w, h) in rects:
6         pts = [(x, y), (x + w, y), (x, y + h), (x + w, y + h)]
7         nps = [(-yy, xx) for (xx, yy) in pts]
8         minx = min(p[0] for p in nps)
9         maxx = max(p[0] for p in nps)
10        miny = min(p[1] for p in nps)
11        maxy = max(p[1] for p in nps)
12        new.append((minx, miny, maxx - minx, maxy - miny))
13    minx = min(r[0] for r in new)
14    miny = min(r[1] for r in new)
15    return [(x - minx, y - miny, w, h) for (x, y, w, h) in new]
16
17
18 def place(shape, perm, rots):
19     """按树形+排列+旋转放置 4 超块，返回（合法？，包围盒面积，每模块 [(
        name, rot, x, y, rects)])"""
20    root, P, L, R = shape
21    placed = []           # 全局已放置矩形 (x, y, w, h, owner)
22    mods = []            # 每模块: (name, rot_idx, x, y, rects_abs)
23    xs = [0] * 4
24    ys = [0] * 4

```

```

25
26 def support(rect):
27     """rect 需要的 y 支撑顶：与已放置矩形 x 区间相交的最大顶。"""
28     top = 0
29     rx0, ry0, rw, _ = rect
30     for (px, py, pw, ph, _) in placed:
31         if px < rx0 + rw and rx0 < px + pw:
32             if py + ph > top:
33                 top = py + ph
34     return top
35
36 def dfs(sid):
37     name = NAMES[perm[sid]]
38     rects = ROTATIONS[name][rots[sid]]
39     if sid == root:
40         xs[sid] = 0
41     else:
42         p = P[sid]
43         pname = NAMES[perm[p]]
44         pw, _ = bbox_of(ROTATIONS[pname][rots[p]])
45         if L[p] == sid:
46             xs[sid] = xs[p] + pw
47         else:
48             xs[sid] = xs[p]
49     # 超块 y = max(内部矩形所需 y)
50     y = 0
51     for (rx, ry, rw, rh) in rects:
52         need = support((xs[sid] + rx, ry, rw, rh)) - ry
53         if need > y:
54             y = need
55     ys[sid] = y
56     for (rx, ry, rw, rh) in rects:
57         placed.append((xs[sid] + rx, y + ry, rw, rh, name))
58     mods.append((name, rots[sid], xs[sid], y, rects))
59     if L[sid] != -1:
60         dfs(L[sid])
61     if R[sid] != -1:
62         dfs(R[sid])
63
64 dfs(root)
65 # 矩形级重叠检查 (6 实际矩形两两)
66 for i in range(len(placed)):
67     for j in range(i + 1, len(placed)):
68         a, b = placed[i], placed[j]
69         if a[0] < b[0] + b[2] and b[0] < a[0] + a[2] and \

```

```
70         a[1] < b[1] + b[3] and b[1] < a[1] + a[3]:
71             return False, 0, None
72     maxx = max(r[0] + r[2] for r in placed)
73     maxy = max(r[1] + r[3] for r in placed)
74     return True, maxx * maxy, mods
```